

Month 5 – Phase 6: DSA + LLD (Days 113–140) – Easy → Hard

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 7 Dec 2026 – 3 Jan 2027 (Days 113–140)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm** & **9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import <calendar/mern-mastery-study.ics>

Master guide: [19-phases-678-easy-to-hard-complete.md](#)

Roadmap: [ROADMAP.md](#)

Sub-phase map

Sub	Days	Difficulty	This PDF section
6A	113–116	● Easy DSA	WEEK 17 (Days 113–116)
6B	117–119	● Easy LLD	WEEK 17B (Days 117–119)
6C	120–123	● Medium DSA	WEEK 18 (Days 120–123)
6D	124–127	● Medium LLD	WEEK 18B (Days 124–127)
6E	128–132	● Med-Hard DSA (DP, intervals, backtracking)	WEEK 19 (Days 128–132)
6F	133–135	● Hard + company-tagged DSA	WEEK 19B (Days 133–135)
6G	136–140	● Backend MC + frontend autocomplete	WEEK 20 (Days 136–140)

Do NOT skip to 6F/6G before completing 6A–6E.

DSA Approach Template (Use for Every Problem)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Copy this skeleton before coding any LeetCode in Month 5:

1. **Restate** problem in one sentence; note input/output constraints.
 2. **Examples** — walk through 2 including edge (empty, single, duplicates).
 3. **Pattern** — name pattern (two pointers, sliding window, etc.).
 4. **Brute force** — time/space complexity.
 5. **Optimized** — outline algorithm in 3–5 bullet steps.
 6. **Code** — implement; name variables clearly.
 7. **Verify** — trace one example on paper.
 8. **Complexity** — state $O()$ formally aloud.
 9. **Follow-ups** — what if streaming input? negative numbers?
-

WEEK 17 – Phase 6A: Easy DSA refresh (Days 113-116)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

6A = Easy only. If a problem feels hard on Day 113, do Easy alternatives first:
LC 167 Two Sum II, LC 125 Valid Palindrome, LC 1 Two Sum, LC 242 Valid Anagram.
LC 15 (3Sum) below is **medium** — save for Day 116 or Phase 6E if 113–115 feel rushed.

Day 113 – LC 167 Two Sum II + LC 125 Valid Palindrome (Easy)

Day 114 – LC 1 Two Sum + LC 242 Valid Anagram (Easy)

Day 115 – LC 20 Valid Parentheses + LC 155 Min Stack (Easy-Medium)

Day 116 – LC 121 Best Time Buy/Sell + LC 283 Move Zeroes OR LC 15 3Sum (stretch)

Problem statement

Return all unique triplets in array summing to zero.

Learning objective

Sort + two pointers; skip duplicates.

Prerequisites

- [08-dsa-patterns-complete.md §1 Two Pointers](#)

Step-by-step approach

1. Sort array ascending.
2. Fix index `i` ; two pointers `l=i+1` , `r=n-1` .

3. Sum compare to 0; move pointers; skip duplicate `i`, `l`, `r` values.
4. Collect triplets; avoid duplicate triplet sets.
5. $O(n^2)$ time, $O(1)$ extra excluding output.

Hints

1. Sorting enables two-pointer on remainder.
2. If `nums[i] > 0` break early.
3. Skip while `nums[l] === nums[l-1]` after finding answer.

Exact Answer

- `[-1,0,1,2,-1,-4] → [[-1,-1,2], [-1,0,1]]`
- Time: $O(n^2)$, Space: $O(1)$ excluding output

Complete Solution

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const result = [];
  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i - 1]) continue;
    if (nums[i] > 0) break;
    let l = i + 1, r = nums.length - 1;
    while (l < r) {
      const sum = nums[i] + nums[l] + nums[r];
      if (sum === 0) {
        result.push([nums[i], nums[l], nums[r]]);
        while (l < r && nums[l] === nums[l + 1]) l++;
        while (l < r && nums[r] === nums[r - 1]) r--;
        l++; r--;
      } else if (sum < 0) l++;
      else r--;
    }
  }
  return result;
}
```

Expected output / acceptance criteria

- `[-1,0,1,2,-1,-4] → [[-1,-1,2], [-1,0,1]]`

Where to practice

- <https://leetcode.com/problems/3sum/>

Common mistakes

- Not skipping duplicate indices → duplicate triplets in result.

- Using triple nested loops on interview without optimizing.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Interview connection

Extension of Two Sum; tests careful duplicate handling.

Time estimate: 45 minutes

LC 11 – Container With Most Water

Problem statement

Max area between two lines (heights array).

Step-by-step approach

1. `left=0` , `right=n-1` , `maxArea=0` .
2. `Area = min(h[l],h[r]) * (r-l)` .
3. Move shorter side inward (greedy proof).
4. Update max each step.

Exact Answer

- `[1,8,6,2,5,4,8,3,7]` → `49` (between indices 1 and 8)
- Time: $O(n)$, Space: $O(1)$

Complete Solution

```
function maxArea(height) {
  let left = 0, right = height.length - 1, max = 0;
  while (left < right) {
    max = Math.max(max, Math.min(height[left], height[right]) * (right - left));
    if (height[left] < height[right]) left++;
    else right--;
  }
  return max;
}
```

Where to practice

- <https://leetcode.com/problems/container-with-most-water/>

Time estimate: 35 minutes

Day 114 – LC 3 + LC 424

LC 3 – Longest Substring Without Repeating

Pattern: Sliding window + Set/Map of last index.

Step-by-step approach

1. `left=0`, map char→index.
2. Expand right; if char seen and `index >= left`, move left to `index+1`.
3. Track max length.

Exact Answer

- `"abcabcbb"` → 3 (`"abc"`); `"bbbbbb"` → 1
- Time: $O(n)$, Space: $O(\min(n, \text{alphabet}))$

Complete Solution

```
function lengthOfLongestSubstring(s) {  
  const map = new Map();  
  let left = 0, max = 0;  
  for (let right = 0; right < s.length; right++) {  
    if (map.has(s[right]) && map.get(s[right]) >= left) {  
      left = map.get(s[right]) + 1;  
    }  
    map.set(s[right], right);  
    max = Math.max(max, right - left + 1);  
  }  
  return max;  
}
```

Where to practice

- <https://leetcode.com/problems/longest-substring-without-repeating-characters/>

80 LPA Interview Twist

Follow-up LC 76 (minimum window) and at-most-K-distinct — Google tests sliding window template mastery across 4+ variants in one round.

Time estimate: 40 minutes

LC 424 – Longest Repeating Character Replacement

Step-by-step approach

1. Window counts; track max freq in window.
2. Valid if `windowSize - maxFreq <= k`.
3. Shrink left when invalid.

Exact Answer

- "AABABBA", $k=1 \rightarrow 4$ ("AABA" or "ABBA")
- Time: $O(n)$, Space: $O(26)$

Complete Solution

```
function characterReplacement(s, k) {  
  const count = {};  
  let left = 0, maxFreq = 0, maxLen = 0;  
  for (let right = 0; right < s.length; right++) {  
    count[s[right]] = (count[s[right]] || 0) + 1;  
    maxFreq = Math.max(maxFreq, count[s[right]]);  
    while (right - left + 1 - maxFreq > k) {  
      count[s[left++]]--;  
    }  
    maxLen = Math.max(maxLen, right - left + 1);  
  }  
  return maxLen;  
}
```

Where to practice

- <https://leetcode.com/problems/longest-repeating-character-replacement/>

Time estimate: 50 minutes

Day 115 – LC 49 + LC 128

LC 49 – Group Anagrams

Step-by-step approach

1. Key = sorted string or 26-char count signature.
2. Map key → list of strings.
3. Return values.

Exact Answer

- `["eat","tea","tan","ate","nat","bat"] → [["eat","tea","ate"],["tan","nat"],["bat"]]`
- Time: $O(n \times k \log k)$, Space: $O(n \times k)$ where k = max string length

Complete Solution

```
function groupAnagrams(strs) {  
  const map = new Map();  
  for (const s of strs) {  
    const key = s.split('').sort().join('');  
    if (!map.has(key)) map.set(key, []);  
    map.get(key).push(s);  
  }  
  return [...map.values()];  
}
```

Where to practice

- <https://leetcode.com/problems/group-anagrams/>

80 LPA Interview Twist

Use 26-char frequency signature instead of sorting for $O(n)$ — Meta asks you to compare hash key strategies and their collision properties.

Time estimate: 30 minutes

LC 128 – Longest Consecutive Sequence

Step-by-step approach

1. Insert all into Set.
2. For each start of sequence (no num-1 in set), walk num+1.
3. $O(n)$ despite nested appearance.

Exact Answer

- `[100,4,200,1,3,2] → 4` (sequence 1,2,3,4)
- Time: $O(n)$, Space: $O(n)$

Complete Solution

```
function longestConsecutive(nums) {  
  const set = new Set(nums);  
  let max = 0;  
  for (const n of set) {  
    if (!set.has(n - 1)) {  
      let len = 1;  
      while (set.has(n + len)) len++;  
      max = Math.max(max, len);  
    }  
  }  
  return max;  
}
```

Where to practice

- <https://leetcode.com/problems/longest-consecutive-sequence/>

Time estimate: 40 minutes

Day 116 – LC 560 + LC 76

LC 560 – Subarray Sum Equals K

Pattern: Prefix sum + hash map count.

Step-by-step approach

1. Map prefixSum → frequency; seed {0:1}.
2. For each prefix, add count of prefix-k to result.

Exact Answer

- Optimal solution with stated time/space complexity
- Passes all LeetCode test cases including edge cases

Complete Solution

```
function coinChange(coins, amount) {  
  const dp = Array(amount + 1).fill(Infinity);  
  dp[0] = 0;  
  for (let a = 1; a <= amount; a++)  
    for (const c of coins)  
      if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);  
  return dp[amount] === Infinity ? -1 : dp[amount];  
}  
// Time O(amount × coins), Space O(amount)
```

Where to practice

- <https://leetcode.com/problems/subarray-sum-equals-k/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 45 minutes

LC 76 – Minimum Window Substring

Step-by-step approach

1. Need map for t char counts; have map tracks window.
2. Expand until valid; shrink to minimize; track best window.

Where to practice

- <https://leetcode.com/problems/minimum-window-substring/>

Time estimate: 60 minutes

Day 117 – Revision Day

Problem statement

Redo 2 weakest problems from Week 17 timed 35 min each without hints.

Exact Answer

- Follow acceptance criteria in Expected output section
- Implementation matches step-by-step approach requirements

Complete Solution

```
function coinChange(coins, amount) {
  const dp = Array(amount + 1).fill(Infinity);
  dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const c of coins)
      if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
  return dp[amount] === Infinity ? -1 : dp[amount];
}
// Time O(amount × coins), Space O(amount)
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 2 hours

Day 118 – Sat: LC 239, 438, 567

Problem	Pattern	Link
LC 239 Sliding Window Maximum	Monotonic deque	https://leetcode.com/problems/sliding-window-maximum/
LC 438 Find All Anagrams	Fixed window + freq	https://leetcode.com/problems/find-all-anagrams-in-a-string/
LC 567 Permutation in String	Same as anagram window	https://leetcode.com/problems/permutation-in-string/

Exact Answer

- LC 239: `[1,3,-1,-3,5,3,6,7]`, `k=3` → `[3,3,5,5,6,7]` — Time $O(n)$, Space $O(k)$
- LC 438: find all anagram start indices — Time $O(n)$, Space $O(1)$
- LC 567: check permutation in string — Time $O(n)$, Space $O(1)$

Complete Solution

```
// LC 239 – Sliding Window Maximum (monotonic deque)
function maxSlidingWindow(nums, k) {
  const deque = [], result = [];
  for (let i = 0; i < nums.length; i++) {
    while (deque.length && deque[0] <= i - k) deque.shift();
    while (deque.length && nums[deque.at(-1)] <= nums[i]) deque.pop();
    deque.push(i);
    if (i >= k - 1) result.push(nums[deque[0]]);
  }
  return result;
}

// LC 438 – Find All Anagrams (fixed window + freq compare)
function findAnagrams(s, p) {
  if (s.length < p.length) return [];
  const need = new Array(26).fill(0), have = new Array(26).fill(0);
  for (const c of p) need[c.charCodeAt(0) - 97]++;
  const result = [];
  for (let i = 0; i < s.length; i++) {
    have[s.charCodeAt(i) - 97]++;
    if (i >= p.length) have[s.charCodeAt(i - p.length) - 97]--;
    if (i >= p.length - 1 && have.every((v, j) => v === need[j])) result.push(i - p.length + 1);
  }
  return result;
}
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 8 hours total day (3 problems + review notes)

Day 119 – Sun: LLD LRU Cache

Problem statement

Design and implement LRU Cache: `get`, `put` $O(1)$ average.

Learning objective

HashMap + doubly linked list; classic LLD interview.

Prerequisites

- [09-ll-d-complete.md § LRU Cache](#)

Step-by-step approach (LLD build — 50 min)

1. **Requirements (10 min):** capacity, get returns -1 if missing, put evicts LRU when full.
2. **Class diagram (10 min):** `LRUCache` , `Node { key, val, prev, next }` , Map key→Node.
3. **Code (25 min):** dummy head/tail; on get move to head; on put update or insert; evict tail.prev.
4. **Explain (5 min):** extensibility to TTL, thread-safe variant.

Hints

1. Map gives O(1) lookup; DLL gives O(1) eviction order.
2. Move-to-front on access maintains recency.
3. Update existing key doesn't change size.

Exact Answer

- `get` and `put` average O(1); evicts least recently used when at capacity
- `get` missing key returns -1

Complete Solution

```
class Node {
  constructor(public key: number, public val: number, public prev: Node | null = null) {}
}

class LRUCache {
  private map = new Map<number, Node>();
  private head = new Node(0, 0);
  private tail = new Node(0, 0);

  constructor(private capacity: number) {
    this.head.next = this.tail;
    this.tail.prev = this.head;
  }

  get(key: number): number {
    if (!this.map.has(key)) return -1;
    const node = this.map.get(key)!;
    this.moveToFront(node);
    return node.val;
  }

  put(key: number, value: number): void {
    if (this.map.has(key)) {
      const node = this.map.get(key)!;
      node.val = value;
      this.moveToFront(node);
    } else {
      const node = new Node(key, value);
      this.map.set(key, node);
      this.addToFront(node);
      if (this.map.size > this.capacity) {
        const lru = this.tail.prev!;
        this.remove(lru);
        this.map.delete(lru.key);
      }
    }
  }

  private moveToFront(node: Node) { this.remove(node); this.addToFront(node); }
  private addToFront(node: Node) {
    node.next = this.head.next; node.prev = this.head;
    this.head.next!.prev = node; this.head.next = node;
  }
  private remove(node: Node) {
    node.prev!.next = node.next; node.next!.prev = node.prev;
  }
}
```

Expected output / acceptance criteria

- LeetCode 146 accepts solution OR unit tests pass custom implementation.

Where to practice

- <https://leetcode.com/problems/lru-cache/>
- Local: `mern-mastery/lld/week-17/lru-cache.ts`

Interview connection

Top LLD problem at Amazon, Microsoft, Uber.

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 8 hours (includes 3 review problems optional)

WEEK 18 – Binary Search, Stack, Linked List (Days 120-126)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Daily DSA Problems

Day	Problems	Approach focus
120	LC 33, 153	Binary search on rotated array; compare mid to bounds
121	LC 74, 875	2D matrix BS; BS on answer space
122	LC 739, 84	Monotonic stack next greater / histogram
123	LC 25, 138	K-group reverse linked list; clone with hash map
124	Revision	2 timed mediums
125 Sat	LC 56, 57, 452	Merge intervals; array minimum
126 Sun	LLD Rate Limiter + Parking Lot	See below

LC 33 – Search Rotated Sorted Array (Detailed)

Step-by-step approach

1. BS while $left \leq right$.
2. Identify sorted half by comparing $nums[left]$ and $nums[mid]$.
3. Check if target in sorted half range; narrow accordingly.

Exact Answer

- `search([4,5,6,7,0,1,2], 0) → 4`
- Time: $O(\log n)$, Space: $O(1)$

Complete Solution

```
function search(nums, target) {  
  let left = 0, right = nums.length - 1;  
  while (left <= right) {  
    const mid = left + Math.floor((right - left) / 2);  
    if (nums[mid] === target) return mid;  
    if (nums[left] <= nums[mid]) {  
      if (nums[left] <= target && target < nums[mid]) right = mid - 1;  
      else left = mid + 1;  
    } else {  
      if (nums[mid] < target && target <= nums[right]) left = mid + 1;  
      else right = mid - 1;  
    }  
  }  
  return -1;  
}
```

Where to practice

- <https://leetcode.com/problems/search-in-rotated-sorted-array/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 40 minutes

Day 126 – LLD: Rate Limiter + Parking Lot

Rate Limiter (Token Bucket)

Step-by-step approach

1. **Entities:** `RateLimiter`, `UserBucket { tokens, lastRefill }`.
2. **Methods:** `allowRequest(userId)` boolean.
3. Refill tokens by elapsed time; cap at max; deduct 1 if allowed.
4. Discuss distributed Redis version.

Exact Answer

- `allowRequest(userId)` returns true if token available, false otherwise
- Tokens refill at configured rate up to max capacity

Complete Solution

```
class TokenBucketRateLimiter {
  private buckets = new Map<string, { tokens: number; lastRefill: number }>();

  constructor(private maxTokens: number, private refillRate: number) {}

  allowRequest(userId: string): boolean {
    const now = Date.now();
    let bucket = this.buckets.get(userId) ?? { tokens: this.maxTokens, lastRefill: 0 };
    const elapsed = (now - bucket.lastRefill) / 1000;
    bucket.tokens = Math.min(this.maxTokens, bucket.tokens + elapsed * this.refillRate);
    bucket.lastRefill = now;
    if (bucket.tokens < 1) { this.buckets.set(userId, bucket); return false; }
    bucket.tokens -= 1;
    this.buckets.set(userId, bucket);
    return true;
  }
}
```

Where to practice

- Local: `l1d/week-18/rate-limiter.ts`

80 LPA Interview Twist

Netflix/Meta use Redis for rate limiting, pub/sub, and session store — expect distributed Redis cluster and failover discussion.

Time estimate: 2 hours

Parking Lot

Step-by-step approach

1. **Entities:** `ParkingLot`, `Level`, `Spot`, `Vehicle` types (car, bike).
2. **Methods:** `park(vehicle)`, `unpark(ticket)`, `findSpot(vehicle)`.
3. Strategy: nearest available spot or smallest fit.
4. Use enums for spot size compatibility.

Exact Answer

- `park(car)` returns ticket ID or null if no compatible spot
- `unpark(ticket)` frees spot; smallest-fit spot assignment strategy

- Supports multiple vehicle types (BIKE, CAR, TRUCK) and spot sizes

Complete Solution

```
enum VehicleType { BIKE, CAR, TRUCK }
enum SpotSize { SMALL, MEDIUM, LARGE }

class Vehicle {
  constructor(public type: VehicleType, public plate: string) {}
}

class ParkingSpot {
  constructor(public id: string, public size: SpotSize, public vehicle: Vehicle | null) {}
  canFit(v: Vehicle): boolean {
    const req = { [VehicleType.BIKE]: SpotSize.SMALL, [VehicleType.CAR]: SpotSize.MEDIUM, [VehicleType.TRUCK]: SpotSize.LARGE };
    return !this.vehicle && this.size >= req[v.type];
  }
}

class Level {
  constructor(public floor: number, public spots: ParkingSpot[]) {}
  findSpot(v: Vehicle): ParkingSpot | null {
    return this.spots.find(s => s.canFit(v)) ?? null;
  }
}

class ParkingLot {
  constructor(private levels: Level[]) {}
  park(vehicle: Vehicle): string | null {
    for (const level of this.levels) {
      const spot = level.findSpot(vehicle);
      if (spot) {
        spot.vehicle = vehicle;
        return `T-${level.floor}-${spot.id}-${Date.now()}`;
      }
    }
    return null;
  }
  unpark(ticket: string): void {
    const [, floor, spotId] = ticket.split("-");
    const level = this.levels.find(l => l.floor === parseInt(floor));
    const spot = level?.spots.find(s => s.id === spotId);
    if (spot) spot.vehicle = null;
  }
}
```

Where to practice

- [09-lld-complete.md](#)

Time estimate: 3 hours

WEEK 19 – Trees + Graphs (Days 127-133)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Day	Problems	Key approach
127	LC 102, 105	BFS level order; build tree from preorder+inorder
128	LC 199, 236	Right side view; LCA recursion
129	LC 200, 133	Grid DFS/BFS islands; clone graph
130	LC 207, 210	Topological sort Kahn's / DFS cycle detect
131	Revision	
132 Sat	LC 127, 130, 417	Word ladder BFS; pacific atlantic
133 Sun	LLD Splitwise + Vending Machine	

LC 200 – Number of Islands (Template)

Step-by-step approach

1. Iterate grid; on '1' start DFS/BFS mark visited sink to '0'.
2. Increment count per new island.
3. 4-directional neighbors.

Exact Answer

- Grid with 4 connected '1's counted as one island; returns total island count
- Time: $O(m \times n)$, Space: $O(m \times n)$ recursion stack

Complete Solution

```
function numIslands(grid) {
  let count = 0;
  const dfs = (r, c) => {
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length || grid[r][c] !==
    grid[r][c] = "0";
    dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1);
  };
  for (let r = 0; r < grid.length; r++)
    for (let c = 0; c < grid[0].length; c++)
      if (grid[r][c] === "1") { count++; dfs(r, c); }
  return count;
}
```

Where to practice

- <https://leetcode.com/problems/number-of-islands/>

80 LPA Interview Twist

Amazon LP interviews use STAR with 'I' statements and dig 3 levels deep on YOUR specific contribution — team answers score poorly.

Time estimate: 35 minutes

Day 133 – LLD Splitwise + Vending Machine

Splitwise approach

1. Entities: User, Expense, Balance sheet (who owes whom).
2. `addExpense(paidBy, amounts[], splitType)`.
3. Simplify debts optional greedy.

Vending Machine approach

1. States: idle, hasMoney, dispensing.
2. Inventory slots; coin accumulator; return change algorithm.

Local: `lld/week-19/`

Exact Answer

- Splitwise: `addExpense(paidBy, splits)` updates balance sheet; optional debt simplification
- Vending Machine: state pattern (IDLE → HAS_MONEY → DISPENSING); returns correct change

Complete Solution

```
// Splitwise
class Splitwise {
  balances = new Map<string, Map<string, number>>>();
  addExpense(paidBy: string, amounts: { user: string; amount: number }[]) {
    for (const { user, amount } of amounts) {
      if (user === paidBy) continue;
      this.addDebt(user, paidBy, amount);
    }
  }
  addDebt(from: string, to: string, amount: number) {
    if (!this.balances.has(from)) this.balances.set(from, new Map());
    const curr = this.balances.get(from)?.get(to) ?? 0;
    this.balances.get(from)?.set(to, curr + amount);
  }
}

// Vending Machine
enum State { IDLE, HAS_MONEY, DISPENSING }
class VendingMachine {
  state = State.IDLE;
  balance = 0;
  insertCoin(cents: number) { this.balance += cents; this.state = State.HAS_MONEY; }
  selectProduct(price: number): boolean {
    if (this.balance >= price) { this.state = State.DISPENSING; this.balance -= price; }
    return false;
  }
}
```

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 8 hours

WEEK 20 – Dynamic Programming (Days 134-140)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Day	Problems	Pattern
134	LC 198, 213	House robber linear / circular
135	LC 322, 518	Coin change min ways / count combinations
136	LC 1143, 72	LCS; edit distance 2D DP
137	LC 312 or 91	Interval DP or decode ways
138	Mixed 5 DP revision	
139 Sat	LC 139, 152, 416	Word break; max product; subset sum
140 Sun	LLD BookMyShow + 200 LC milestone	

DP Approach Template Extension

1. Define state `dp[i]` or `dp[i][j]` meaning in English.
2. Base cases explicitly.
3. Recurrence relation on paper with example.
4. Iteration order (forward/backward).
5. Space optimization if only previous row needed.

LC 322 – Coin Change (Detailed)

Step-by-step approach

1. `dp[0]=0 ; dp[amount]=min(dp[amount-coin]+1) .`
2. Iterate amount 1..target; inner loop coins.
3. If unreachable stay infinity; return -1 if `dp[amount]` inf.

Exact Answer

- `coinChange([1,2,5], 11) → 3 (5+5+1)`
- Time: $O(\text{amount} \times \text{coins})$, Space: $O(\text{amount})$

Complete Solution

```
function coinChange(coins, amount) {  
  const dp = Array(amount + 1).fill(Infinity);  
  dp[0] = 0;  
  for (let a = 1; a <= amount; a++) {  
    for (const coin of coins) {  
      if (coin <= a) dp[a] = Math.min(dp[a], dp[a - coin] + 1);  
    }  
  }  
  return dp[amount] === Infinity ? -1 : dp[amount];  
}
```

Where to practice

- <https://leetcode.com/problems/coin-change/>

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 45 minutes

Day 140 – LLD BookMyShow

Problem statement

Seat booking with concurrency: show, seats, lock during booking, payment confirm.

Step-by-step approach

1. **Entities:** Movie, Show, Screen, Seat (status available/held/booked).
2. **Flow:** select seats → hold 5 min → pay → confirm or release.
3. In-memory lock map showId+seatId; handle double booking.
4. Discuss scale to distributed locks.

Exact Answer

- Seat hold for 5 minutes during booking; concurrent double-booking prevented via lock map
- Flow: select → hold → pay → confirm (or release hold on timeout)

Complete Solution

```
enum SeatStatus { AVAILABLE, HELD, BOOKED }

class BookMyShow {
  private seats = new Map<string, SeatStatus>();
  private holds = new Map<string, { seats: string[]; expiresAt: number }>();

  holdSeats(showId: string, seatIds: string[], userId: string): string | null {
    for (const id of seatIds) {
      if (this.seats.get(`${showId}:${id}`) !== SeatStatus.AVAILABLE) return null;
    }
    for (const id of seatIds) this.seats.set(`${showId}:${id}`, SeatStatus.HELD);
    const holdId = crypto.randomUUID();
    this.holds.set(holdId, { seats: seatIds.map(id => `${showId}:${id}`), expiresAt: Date.now() + 60 * 60 * 1000 });
    return holdId;
  }

  confirmBooking(holdId: string): boolean {
    const hold = this.holds.get(holdId);
    if (!hold || Date.now() > hold.expiresAt) return false;
    for (const key of hold.seats) this.seats.set(key, SeatStatus.BOOKED);
    this.holds.delete(holdId);
    return true;
  }
}
```

Where to practice

- Local: `l1d/week-20/bookmyshow.ts`

80 LPA Interview Twist

At Google/Meta 80 LPA level, expect follow-up questions on scale (10× traffic), failure modes, and production tradeoffs — not just happy-path implementation.

Time estimate: 6 hours

LLD Session Rules (All Weeks)

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

Phase	Minutes	Activity
1	10	Requirements + entities + API methods
2	10	Class diagram on paper
3	25	TypeScript core classes
4	5	Extensibility aloud (plugins, scaling)

Where to code: `mern-mastery/lld/week-XX/`

Month 5 Completion Checklist

Milestone	Target	Done
LeetCode total	200+	[]
Medium solve time	≤35 min avg	[]
LLD problems coded	6 (LRU, Rate Limit, Parking, Splitwise, Vending, BookMyShow)	[]
Pattern notes	One page per pattern	[]

Total estimated time: ~80–96 hours

Primary practice URL: <https://leetcode.com/problemset/>